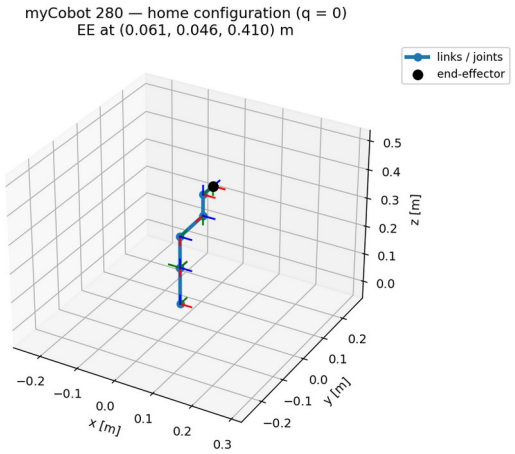
# R_sine — Drawing a Sine Wave with a myCobot 280

Analytical kinematics + inverse kinematics on a 6-DOF arm (ROS 2)



# Objective and Method

- Forward kinematics: where is the tool, given the joint angles?
- Jacobian: how tool velocity relates to joint velocities
- Inverse kinematics: which joint angles reach a desired tool pose?
- Pipeline: sine path -> IK per point -> joint trajectory -> RViz

# Modified (Craig) DH Convention

*Four parameters relate each pair of link frames*

- alpha_(i-1): link twist — angle between successive joint axes
- a_(i-1): link length — offset between successive joint axes
- d_i: link offset along the joint axis
- theta_i: joint angle (the variable for a revolute joint)
- Link transform: Rotx(alpha)·Transx(a)·Rotz(theta)·Transz(d)

# Identified DH Table (verified to 1e-15 m)

*Fitted from the URDF, then checked against it*

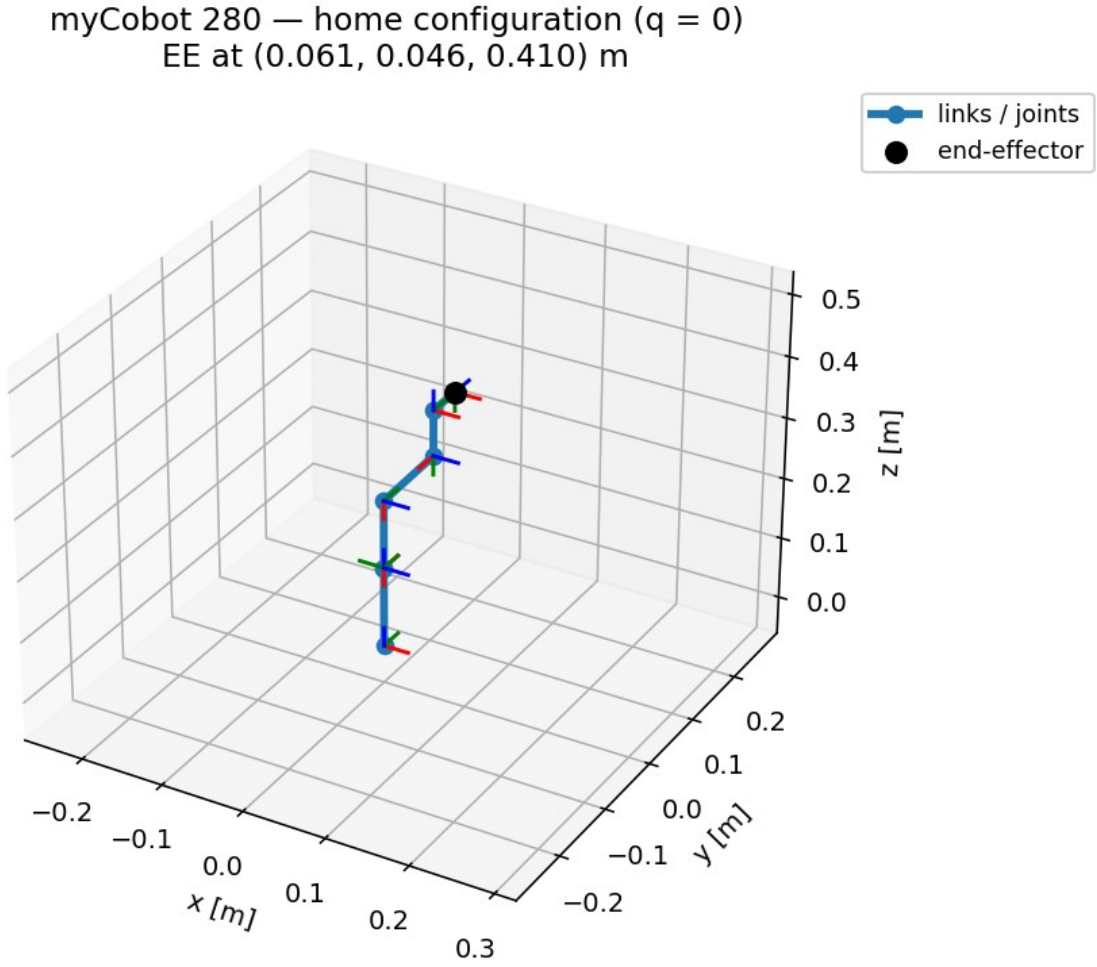
- URDF link frames are not on DH axes -> table was identified numerically
- Fit (alpha, a, d, theta-offset) so DH-FK reproduces URDF-FK
- Residual is machine-zero: max |DH-FK - URDF-FK| ~ 1e-15 m
- Physically clean 'concentrated' convention chosen for readability

| i | alpha (deg) | a (m) | d (m) | theta off (deg) |
|---|---|---|---|---|
| 1 | 0 | 0 | 0.13056 | +90 |
| 2 | +90 | 0 | 0 | -90 |
| 3 | 0 | -0.1104 | 0 | 0 |
| 4 | 0 | -0.096 | 0.06062 | -90 |
| 5 | +90 | 0 | 0.07318 | +90 |
| 6 | -90 | 0 | 0.0456 | 0 |

# Forward Kinematics

*Product of the six link transforms*

- T_EE = T1·T2·T3·T4·T5·T6
- Home pose (all joints 0): tool at (0.061, 0.046, 0.410) m
- Independently confirmed by ikpy's FK (~1e-16 m)



# Jacobian — Velocity Propagation

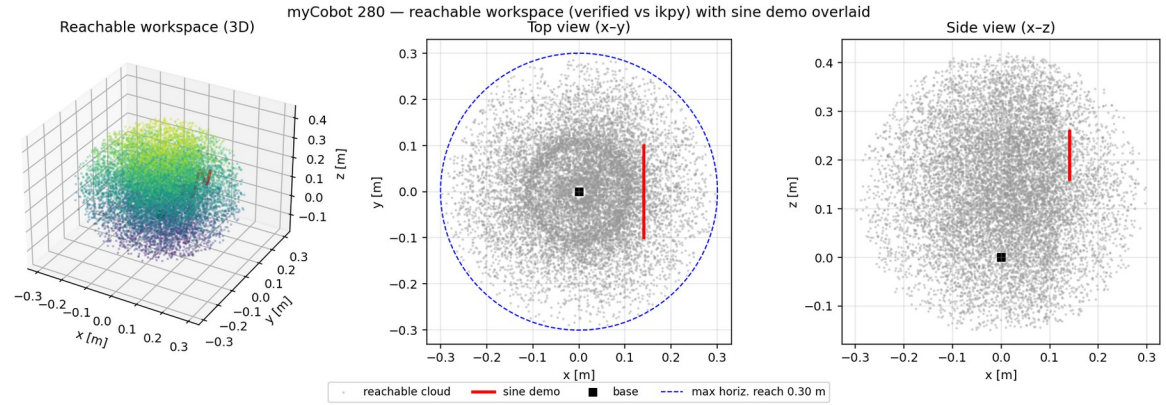
*Built frame-by-frame, verified against finite differences*

- Propagate angular & linear velocity outward: base -> tool
- For revolute joints reduces to columns [ z_i x (p_e - p_i) ; z_i ]
- Maps joint rates to tool twist: [v; omega] = J(q) q_dot
- Verified vs numerical Jacobian to ~2.5e-7

# Reachable Workspace (verified)

*Where the tool can go — the sine sits well inside it*

- Sampled by FK over 40,000 valid joint configurations
- Max 3D reach 0.427 m; max horizontal reach 0.301 m
- Cross-checked against ikpy FK to ~1.7e-16 m
- The demo sine lies safely inside the reachable set



# Inverse Kinematics — Damped Least Squares

*Jacobian-based, robust near singularities*

- q_(k+1) = q_k + J^T (J J^T + lambda^2 I)^(-1) e
- e = [position error ; orientation error] (6-vector)
- Damping lambda keeps updates stable near singularities
- ~16 iterations per waypoint; solutions clamped to joint limits

# Configurable Drawing Plane

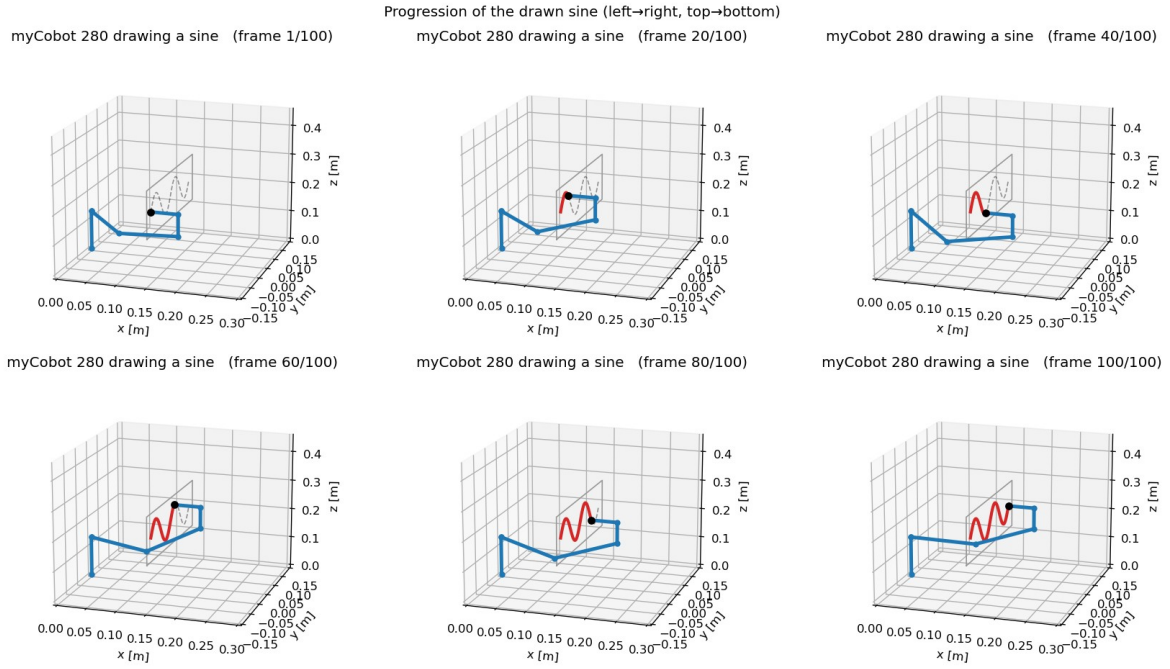
*Same code, any surface*

- Plane defined by origin + two in-plane axes (advance u, wave v)
- Pen kept perpendicular to the surface (flange z along the normal)
- Solver auto-picks the reachable pen direction (+/- normal)
- Vertical board or flat plate — only the parameters change

# Result — Drawing the Sine

*Tool traces the target to sub-millimetre accuracy*

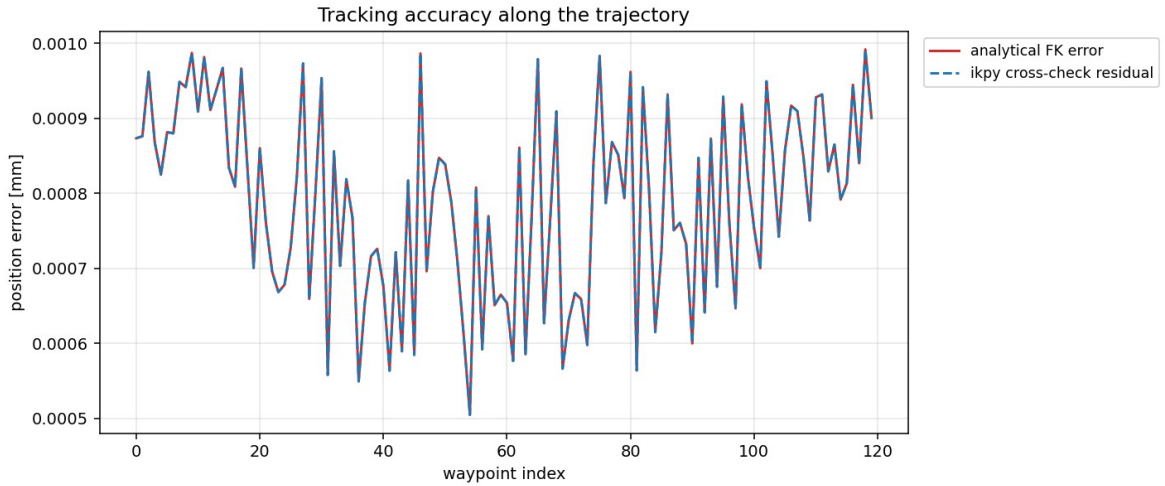
- 20 cm wide, 10 cm tall, 2-cycle sine on a vertical board
- Pen held perpendicular to the board throughout
- Drawn path overlays the target essentially perfectly



# Accuracy and Verification

*Numbers behind the result*

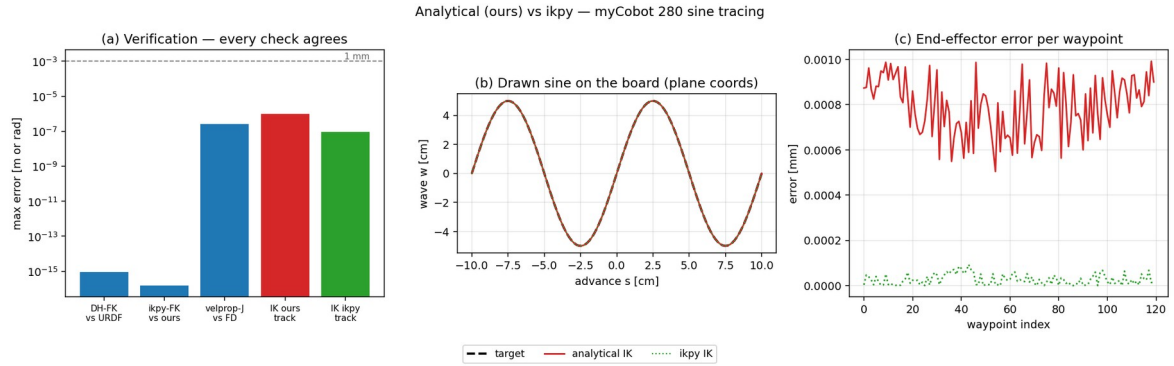
- DH-FK vs URDF-FK: ~1e-15 m
- Velocity-propagation Jacobian vs finite differences: ~2.5e-7
- Analytical IK tracking error: ~0.001 mm
- ikpy cross-check on every waypoint: ~0.001 mm



# Analytical vs ikpy

*Independent methods, same answer*

- My analytical FK/Jacobian/IK vs the ikpy library
- All agreement checks are far below 1 mm (log scale)
- Both solvers trace the same sine



# ROS 2 Architecture

*Clean, separate Python nodes*

- kinematics.py / ik.py / sine_path.py — pure math, no ROS
- sine_ik_node — precomputes IK, streams /joint_states at 30 Hz
- path_tracer_node — accumulates the real drawn line from TF
- One launch file -> robot_state_publisher + nodes + RViz

# Conclusion and Future Work

- First-principles kinematics, fully verified, drawing a sine in ROS 2
- General plane + shape: extensible beyond a sine on a board
- Next: run on the physical myCobot 280; add a real pen and contact
- Next: Gazebo physics; trajectory timing / velocity limits